

FireflySynch v1

Introduction

Goal: Realizzazione in Java del [Fireflysynch](#) limitatamente al requisito 1

Requirements

Costruire il simulatore di un sistema costituito da 3 lucciole. Inizialmente, le lucciole lampeggiamo ciascuna con una frequenza (costante) propria.

- dopo 10 secondi le lucciole devono lampeggiare in modo sincronizzato
- il sistema è dotato anche di un sonar . Quando il sonar rileva un ostacolo a una distanza minore di una distanza prefissata DMIN, le lucciole devono lampeggiare in modo sincronizzato. Quando la distanza dell’ostacolo (ri)supera DMIN, le lucciole tornano a lampeggiare in modo random

Requirement analysis

L'analisi dei requisiti mira a chiarire le entità coinvolte e i vincoli temporali del sistema per il primo requisito.

- Firefly (Lucciola):** Entità autonoma e proattiva capace di emettere un segnale luminoso (flash). Inizialmente ha un periodo T_i costante ma differente per ogni istanza, scelto casualmente.
- Sincronizzazione:** Stato collettivo in cui tutte le lucciole emettono il flash simultaneamente, condividendo frequenza e fase.
- Trigger Temporale:** La transizione dal comportamento asincrono a quello sincronizzato deve avvenire esattamente dopo 10.000 ms.
- Segnale di Sincronizzazione:** Formalizzato come un **Evento** (`Event sync`). Si sceglie l'evento in quanto stimolo ambientale broadcast che non richiede all'emittente la conoscenza puntuale dei destinatari (disaccoppiamento).

Problem analysis

L'analisi si focalizza sulla transizione da una logica proattiva autonoma a una reattiva coordinata, evidenziando le seguenti criticità tecniche:

Architettura Logica

Il [sistema](#) viene modellato tramite il metamodello QAK come un insieme di attori:

- Creator:** Attore coordinatore responsabile della generazione dinamica delle lucciole e della gestione del countdown di 10 secondi. Al termine, emette l'evento globale.
- Firefly:** Attore reattivo (`dynamicOnly`) che implementa una FSM sensibile a timer interni (timeout) ed eventi esterni.

Sincronizzazione di Fase

Un problema rilevato è che impostare lo stesso periodo non garantisce la sincronia visiva se le lucciole si trovano in punti diversi del loro ciclo di attesa locale.

Soluzione: La ricezione dell'evento `sync` deve causare una preemption sul timer locale. L'attore deve interrompere il `delay` in corso e saltare immediatamente allo stato di `flash`. In questo modo si allinea istantaneamente la fase di tutti i componenti.

Formalizzazione dei Messaggi

```
Event sync : sync(PERIOD)           // Segnale broadcast di sincronizzazione
Dispatch cellstate : cellstate(X,Y,C) // Aggiornamento GUI esterna
```

Logica delle Transizioni

La lucciola gestisce la duplice natura del comportamento (autonomo vs coordinato) tramite la seguente struttura di transizione:

```
Transition t0
  whenTimeVar Timer -> flash    // Percorso proattivo (fase asincrona)
  whenEvent sync   -> handleSync // Percorso reattivo (fase sincronizzata)
```

Test plans

Project

Testing

Deployment

Maintenance

